

BACHELOR'S THESIS COMPUTING SCIENCE

Evaluating and Validating the Information Extraction of a Music Newsletter LLM

Subtitle if you like

RUBEN KOELEWIJN
s1105300

May 8, 2026

First supervisor/assessor:
Title(s) Firstname Lastname

Second assessor:
Title(s) Firstname Lastname

Radboud University



Abstract

Brief outline of research questions, results. (The preferred size of an abstract is one paragraph or one page of text.)

Contents

1	Introduction	3
2	Preliminaries	4
2.1	Hallucination in Large Language Models	4
2.2	Cross-Lingual Natural Language Inference	5
3	Research	6
3.1	Pipeline Setup	6
3.1.1	Loading Text	6
3.1.2	Data Loader	6
3.1.3	Decomposer	6
3.1.4	Dutch Parser	6
3.1.5	Validator	6
3.1.6	Classification	7
4	Cloudspeakers Concert Pipeline Validation	8
4.1	Cloudspeakers Experiment Setup	8
4.1.1	Cloudspeakers Pipeline Architecture	8
4.1.2	Newsletter Examples	9
4.2	Cloudspeakers Validation Results	10
5	Case 2	11
6	Related Work	12
7	Conclusions	13
A	Appendix	15

Todo list

	if i decide to use	4
	Give probabilities of LLMs hallucinating to show relevance	5
	Change Title	6
	move footnote to introduction.	6
	find the studies that show LLM can verify themselves.	6
	add table with entailment Guidelines	7
	Psuedocode for to show the for loops and explain complexity?	7
	Back up claim that one sentence is better than full text.	7
	Classification could be an option	7
	A bit better?	8
	Add no of experiments and dates.	9
	add how i do the text extraction when finished.	10

Chapter 1

Introduction

The introduction of your bachelor thesis introduces the research area, the research hypothesis, and the scientific contributions of your work. A good narrative structure is the one suggested by Simon Peyton Jones [?]:

- describe the problem / research question
- motivate why this problem must be solved
- demonstrate that a (new) solution is needed
- explain the intuition behind your solution
- motivate why / how your solution solves the problem (this is technical)
- explain how it compares with related work

Close the introduction with a paragraph in which the content of the next chapters is briefly mentioned (one sentence per chapter).

Research Question LLMs generate text, not always true. Hallucination -i sources.

- *"To what extent can a deterministic validation pipeline—utilizing the Open Web Index and Cross-Lingual NLI (XLM-RoBERTa)—accurately detect factual hallucinations in LLM-generated English summaries of Dutch concert web pages?"*

Motivation

Chapter 2

Preliminaries

This *optional* chapter contains the stuff that your reader needs to know in order to understand your work. Your “audience” consists of fellow third year computing science bachelor students who have done the same core courses as you have, but not necessarily the same specialization, minor, or free electives.

You do not have to call this chapter *Preliminaries* and it does not need to be limited to one chapter. Some preliminary topics deserve their own chapter.

Preliminaries:

- LLM Hallucination
- Cross-Lingual Natural Language Inference (NLI)
- Open Web Index (OWI) Architecture

if i decide
to use

2.1 Hallucination in Large Language Models

In this thesis, we will be looking at the output of different Large Language Models (LLM). LLMs are massive artificial neural networks pre-trained on vast amounts of text data, designed to understand and generate human language. They show strong understanding in natural language and solving complex tasks. [7] While these outputs often contain useful and correct information, they are prone to *hallucinating*. Hallucination of a LLM consists of the model generating content that is not faithful to its source. There exist multiple types of hallucinating, and following the taxonomy proposed by [5], we can categorise these into two types:

- **Factuality Hallucinations**

Discrepancies between generated content and verifiable real-world facts. This includes *Fabrications* (made-up claims) and *Contradictions* (statements that directly oppose reality).

- **Faithfulness Hallucinations**

Divergence from the provided user input or context. This includes *Instruction* (following the wrong instructions from the prompt), *Context* (Not understanding the context) and *Logical* (Incorrect logic, e.g. math errors). In information extraction (IE) tasks, these often manifest as “Missing Spans” or “Incorrect Types” [4].

Classifying the LLM Hallucination, allows us to have insight on what went wrong and where to improve. It gives us the ability to do a system-level diagnosis of the pipeline [6]. For instance,

a *faithfulness* error indicates a failure to maintain “context-boundary” stability [6], where a *factuality* error might need external grounding through retrieval. Because LLMs are often unable to faithfully detect their own factual errors [2], deterministic verification is required to ensure airtight verification.

Give probabilities of LLMs hallucinating to show relevance

2.2 Cross-Lingual Natural Language Inference

Natural Language Inference (NLI), also known as Recognizing Textual Entailment (RTE), is a core task in Natural Language Processing (NLP) [3]. NLP is a multidisciplinary field at the crossover between Computing Science, Linguistics, and Artificial Intelligence, where scientists focus on helping computers understand natural language. Common applications consist of machine translation, sentiment analysis, and automated summarisation. The latter is highly relevant for this thesis, since we deal with Information Extraction (IE), specifically extracting text and entities from websites [4], and Semantic Validation, which involves logically validating the semantic meaning of different sentences to detect errors [5, 2].

The task of NLI in NLP is determining the logical relationship between two pieces of text: a *premise* and a *hypothesis* [3]. Based on the truth of the premise, the model assigns one of three categorical labels as demonstrated in Table 2.1:

Table 2.1: Examples of Natural Language Inference (NLI) relationships.

Premise	Hypothesis	NLI Label
Frank Boeijen is from Nijmegen.	Nijmegen is the hometown of Frank Boeijen.	<i>Entailment</i>
Frank Boeijen is from Nijmegen.	Frank Boeijen is from Amsterdam.	<i>Contradiction</i>
Frank Boeijen is from Nijmegen.	Frank Boeijen is coming back to Nijmegen.	<i>Neutral</i>

- **Entailment:** The hypothesis is guaranteed to be true if the premise is true. In Table 2.1, the premise logically necessitates that Nijmegen is Boeijen’s hometown [3].
- **Contradiction:** The hypothesis is guaranteed to be false if the premise is true. Since the premise establishes Nijmegen as the origin, a claim of Amsterdam constitutes a direct contradiction [3].
- **Neutral:** The truth of the hypothesis cannot be determined from the premise alone. While Frank Boeijen may be “coming back” to Nijmegen, the original premise regarding his origin does not provide sufficient information to confirm or deny a return trip [3].

In the context of this thesis, we utilize *Cross-Lingual* NLI to bridge the language gap between Dutch source text and English generated summaries. By using a multilingual model like XLM-RoBERTa, we can evaluate a Dutch premise against an English hypothesis without an intermediate translation step, preserving the semantic nuances of the original concert metadata [3].

Chapter 3

Research

This chapter, or series of chapters, delves into all technical details that are required to *prove* your scientific hypothesis. Please do not call this chapter or these chapters *Research* as that name is not descriptive. It should be sufficiently detailed and precise in order for any fellow computing scientist student to be able to *repeat* your research and therewith establish the same results / conclusions that you have obtained. Please note that, in order to improve readability of your thesis, you can put a part of this information also in one or more appendices (see Appendix A).

Change
Title

3.1 Pipeline Setup

3.1.1 Loading Text

3.1.2 Data Loader

3.1.3 Decomposer

3.1.4 Dutch Parser

3.1.5 Validator

Theory

Validating the claims is the core part of the operation we are doing. The validation is a process of cross-language NLI, comparing the original Dutch text to the English summaries that are given.

To avoid the inherent bias of self-validation—where an LLM acts as an uncritical judge of its own generated output ¹, the option of using a generative model for the entailment check was discarded. **Even though multiple studies have shown this to be effective, we wish to find a way to do this in a deterministic way due to LLMs having the nature of being unpredictable and prone to hallucination, as mentioned in section 2.1.** Comparing the premise and the hypothesis can be done in multiple ways, but for this thesis we have made the choice to make use of Facebook’s XLM-RoBERTa [3].

XLM-RoBERTa, is a Transformer-based language model trained on 2.5 Terabytes of filtered CommonCrawl data across 100 different languages. XLM-R learns a shared mathematical representation of human language, which results in it being able to convert multiple languages into

move foot-
note to
introduc-
tion.

find the
stud-
ies that
show LLM
can ver-
ify them-
selves.

¹In the Netherlands, this specific conflict of interest is known as acting such as the company ‘WC-Eend’, referencing a famous advertising campaign in which the brand recommends its own product (Wij van Wc-Eend adviseren Wc-Eend).

a mapped mathematical vector space, meaning different languages can be compared. Conneau et al. show that XLM-R is very competitive with strong monolingual models on XNLI benchmarks, demonstrating that it is possible to have a single large model for all languages. [3] For the Cloudspeakers Experiment (section 4), this saves us steps from translating between Dutch and English, or limiting the experiments to pages that *do* have an English variant. As mentioned in 2.2, NLI looks at the entailment, contradiction and neutral scores. XLM-R outputs these when given a premise and hypothesis and using these we can determine likelihood of a hallucination by the LLM.

Guidelines to the entailment scores can be found in Table XX.

INSERT TABLE HERE

Implementation

The `validator` is implemented in Python utilizing PyTorch and the HuggingFace `transformers` library, using the specific `joeddav/xlm-roberta-large-xnli` model.

As mentioned in section XX, the pipeline delivers multiple different claims $\mathcal{C}$ and the original text from the source. Since XLM-R is set up to only check one premise against a single hypothesis, we will run the model for every claim and every sentence in the source material and find which sentence results in the highest entailment and which in the highest contradiction. We chose to run the experiments in this way, since it showed the best results in testing compared to supplying the model with the entire source text at once. For any given claim ($c \in \mathcal{C}$) and a set of source sentences ($\mathcal{S}$), the system iterates through $\mathcal{S}$, tokenizing the premise-hypothesis pairs with trunaction enabled. The raw logits output by the model are passed through a Softmax function to calculate a probability distribution across the earlier mentions classes: **contradiction**, **neutral** and **entailment**. The script holds two trackers for each claim:

1. **max_entailment**: The sentence in $\mathcal{S}$ that returns the highest *entailment* probability
2. **max_contradiction**: The sentence in $\mathcal{S}$ that returns the highest *contradiction* probability.

Testing results can be found in Table XX. These tests showed that XLM-R struggles when the premise and hypothesis aren't of the same length, most likely due to too much information being fed and flooding it with information, leading to a more neutral score. These results are back up by SOURCE HERE!!!

3.1.6 Classification

add table
with en-
tailment
Guidelines

Psuedocode
for to show
the for
loops and
explain
complex-
ity?

Back up
claim that
one sen-
tence is
better
than full
text.

Classification
could be
an option

Chapter 4

Cloudspeakers Concert Pipeline Validation

This part of the thesis will be about the case of the Doornroosje Newsletter.

The first case we will be using our program on is a specific newsletter, which extracts data using Deepseek, a LLM, to summarise current events at Doornroosje, a venue located in Nijmegen.

4.1 Cloudspeakers Experiment Setup

A bit better?

4.1.1 Cloudspeakers Pipeline Architecture

Cloudspeakers is an Concert Event Pipeline setup by Chris Bol. It is an automated pipeline that collects concert events from Dutch music venues, extracts structured data using a Large Language Model, and distributes a weekly email newsletter to subscribers. [1] The system is focused on Utrecht-based venues such as ACU, Ekko, De Helling, db's, De Nijverheid, and Tivoli Vredenburg. The pipeline runs on Windmill, an open-source workflow automation platform. All logic is written in Python and SQL. No manual data entry is required.

The Pipeline looks as follows:

1. Scrape event URLs from venue calendar pages
2. Store scraped pages (HTML + Markdown) in MinIO object storage
3. **LLM extracts structured event data (Deepseek)**
4. Structured data stored in PostgreSQL
5. Email newsletter to subscribers

Feedback is collected from users via a web form and stored in PostgreSQL.

For this thesis, step 3 is most relevant since it uses an LLM to create a summary of the event and the core intelligence step. The fields extracted by the LLM can be found in table 4.1. As seen in table 4.1, the LLM generates an one-sentence English summary including the origin of the artist. This shows that we will be doing Cross-lingual processing, as mentioned in section 2.2 since the website of Doornroosje is mostly in Dutch.

Table 4.1: Description of extracted event metadata fields.

Field	Description
event	Name of the event
venue_name	Name of the venue (or hall)
main_act	Main performing artist or band
support_acts	Support acts (comma-separated)
date	Event date in YYYY-MM-DD format
date_sale	Ticket sale start date
url_sale	Ticket purchase URL
event_type	Category: gig, club night, open podium, talk, film, show, exhibition, other_category
genres	Music genres (comma-separated, lowercase)
summary	One-sentence English summary including artist origin
related_bands	Similar artists mentioned on the page

4.1.2 Newsletter Examples

For our tests, we will be using 30 different output examples, collected from DATE until DATE.

Add no
of experi-
ments and
dates.

```

1  act: First Aid Kit - Visions of the Past Tour
2  type: gig
3  date: 2026-12-02
4  url: https://doornroosje.nl/event/first-aid-kit
5  venue: De Vereeniging
6  bands: First Aid Kit (main)
7  genres: folk, blues, country
8  summary: First Aid Kit, the Swedish duo of sisters Johanna and Klara
           Soderberg, perform their intimate folk music on their Visions of
           the Past Tour.

```

Listing 4.1: Example of a newsletter snippet.

Since current scope of the thesis does not include validating the `type`, `date`, `venue`, `genres`, we do not need to store these. For every entry of the newsletter we validate, remove those entries. The final newsletter is stored as a JSON object, ensuring the validation pipeline can isolate the summary from the core entities. Listing 4.2 shows a complete generated entry.

```

1  {
2    "event": "Daoud Live",
3    "venue_name": "Doornroosje",
4    "main_act": "Daoud",
5    "genres": ["jazz", "hip-hop"],
6    "summary": "French trumpeter Daoud is bringing his unique blend of jazz
7               and hip-hop to Nijmegen for the first time."
8  }

```

Listing 4.2: Example of a simplified newsletter entry.

All examples can be found in Appendix A.

Next, we [as seen in section, we extract the text from the url](#). The extracted text contains all information that is found on the Doornroosje website, giving our pipeline all information that could verify the claims made by the LLM in the summary. This is then stored in a different JSON object (Listing 4.3).

```
1  "VROEGZAT": {  
2    "act": "VroegZat",  
3    "url": "https://www.doornroosje.nl/event/vroegzat-10/",  
4    "text": "VroegZat brengt het nachtleven terug naar de avond!  
           Evenveel feest, maar eerder op de avond (19.30 tot 00.00 uur).  
           Dus op een normale tijd naar bed, waardoor je de volgende dag  
           gewoon fit bent. Win-win! Muzikaal gaat het heen en weer: van  
           disco naar pop, van reggae naar rock en van happy naar hiphop  
           , en die lekkere dansbare tunes die daartussen en daarbuiten  
           vallen. DJ Rob de Nice en MC Rick Rossig gooien alles in de  
           mix. Zorg dat je op tijd binnen bent, want hier kun je niet  
           vroeg genoeg hard gaan!"  
5  }
```

add how i
do the text
extraction
when fin-
ished.

Listing 4.3: Example of extracted text.

4.2 Cloudspeakers Validation Results

Chapter 5

Case 2

This chapter will be about the second experiment.

Chapter 6

Related Work

In this chapter you demonstrate that you are sufficiently aware of the state-of-art knowledge of the problem domain that you have investigated as well as demonstrating that you have found a *new* solution / approach / method.

Chapter 7

Conclusions

In this chapter you present all conclusions that can be drawn from the preceding chapters. It should not introduce new experiments, theories, investigations, etc.: these should have been written down earlier in the thesis. Therefore, conclusions can be brief and to the point.

Bibliography

- [1] Chris Bol. Cloudspeakers - concert event pipeline. Internal technical documentation. Provided directly by the author., March 2026.
- [2] shiqi chen, Yiran Zhao, Jinghan Zhang, I-Chun Chern, Siyang Gao, Pengfei Liu, and Junxian He. Felm: Benchmarking factuality evaluation of large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 44502–44523. Curran Associates, Inc., 2023.
- [3] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.
- [4] Jianhao Gao and et al. An empirical study on information extraction using large language models. *arXiv preprint arXiv:2305.14450*, 2024.
- [5] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.*, 43(2), 2025.
- [6] Vaishali Vinay. Failure modes in llm systems: A system-level taxonomy for reliable ai applications, 2025.
- [7] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.

Appendix A

Appendix

Appendices are *optional* chapters in which you cover additional material that is required to support your hypothesis, experiments, measurements, conclusions, etc. that would otherwise clutter the presentation of your research.